

Calendar handwriting digitizer with content categorization

Antonio Gagliardi¹

Abstract

In the current era of advanced technology, most people take the advantage of using digital calendars accessible from various devices, including laptops, tablets, and smartphones, to record personal commitments. These digital tools include also automated reminders, enhancing time management and organizational efficiency. However, a segment of the population, particularly the elderly, may lack the ability required to use these tools, continuing to rely on paper-based solutions. This technological gap can hinder an effective time management and increase the risk of missed appointments, which may negatively impact the well-being of the person. This study proposes the development of an application designed to scan the model of a physical calendar and convert it into digital format. The tool aims to bridge the technological gap for a segment of people, thereby entrusting the reminders of the commitment management also to someone else, enabling a more effective commitment management, reducing the likelihood of missed appointments and potentially improving the overall well-being of the person.

Keywords

OCR: Optical Characters Recognition, **CNN:** Convolutional Neural Network, **FC:** Fully Connected, **ResNet:** Residual Network, **ViT:** Visual Transformer **NLP:** Natural Language Processing, **BERT:** Bidirectional Encoder Representations from Transformers

HIGHLIGHTS

- Fill a gap in the bibliography by proposing a test with a more real use case.
- Create a pipeline with several neural blocks to tackle the problem with a "divide et impera" approach.
- Explore the potential of Vision Transformer combined with a word pre-segmentation step.
- Attempt of using BERT as a decoder to map word features at the single characters level.
- Exploring the use of a LLM such as BERT to categorize digitized text.

1. Introduction

OCR, Optical Character Recognition, is a wide range of applications dedicated to the detection of characters contained in a document and their conversion into digital text, readable, editable and searchable on a machine. The process is usually carried out by acquiring the image of the document with a digital camera or a scanner. The ability of converting various forms of documents can be applied in a wide range of fields, for example the recognition of human handwriting, the digital conversion of labels and manuscripts, the recognition of numerical digits in financial and banking contexts and the validation of a particular type of handwriting to authenticate the provenance of a manuscript. Another interesting application that has been developed in recent years involves

the recognition of ancient characters [1]. The discovery leads their authors to win the "Vesuvio Challenge"¹. The objective of the challenge was to digital reconstruct ancient damaged papyrus scrolls. The scrolls were digitally "unwrapped" using computed tomography (CT) and machine-learning technology. The resulting scans were then turned into a 3D volume of voxels, which have been segmented by tracing the crumpled layers of the rolled papyrus in the 3D scan, actually flattening the images. The last step was detecting ink on papyrus by using machine learning to identify regions of ink in the flattened segments of the papyrus. What is the most interesting part of this application is that the model has no previous knowledge of character alphabets or handwriting analysis. The digital characters predicted by the model, result therefore purely from plotting the local ink detection spots across a generated image. Another important application of handwriting recognition has been applied to cuneiform tablets [2]. Researchers, instead of using photos, relies on 3D models of the tablets, delivering significantly more reliable results than previous methods. This makes it possible to search through the content of multiple tablets and to compare them with each other. They used 3D models of nearly 2000 cuneiform tablets, many of them are more than 5000 years old and are thus among mankind's oldest surviving written records. What they discover is an extremely wide range of topics, from shopping lists to court rulings, providing a glimpse into mankind's past several millennia ago. However, despite the result obtained, the challenge remains open since there are lot of complications, mostly the fact that some tablets are heavily ruined and also that the writing system was very complex at that age and encompassed several

 gagliardi.1977681@studenti.uniroma1.it (A. Gagliardi)
 <https://github.com/antoogagliardi> (A. Gagliardi)
 © 2021 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).
 CEUR Workshop Proceedings (CEUR-WS.org)

¹<https://scrollprize.org>

languages. Therefore, the model requires not only better data but also more sophisticated background knowledge to capture the different facets of the cuneiform writing.

It is noteworthy that also a major technology company such as Google has developed its own personal OCR system in the more recent years. Google OCR, developed by Google AI [3], is designed to convert a variety of document types, including scanned documents, PDFs, and images captured by a digital camera, into editable text. The system's principal advantages are its high degree of accuracy, achieved through the use of sophisticated deep learning techniques for the recognition and extraction of text with remarkable precision (even in the presence of complex backgrounds or low-quality images), the incorporation of multiple languages within the system leads to the capability of processing a wide range of alphabets, including ideograms, and possibility to process not only printed characters but also handwritten texts.

OCR system plays also an important social role when it is employed in all that applications to deal with visual impairments, giving the possibility to blind people to convert a written text into audio (OCR + speech synthesis). It can be concluded the introductory section highlighting the fact that OCR is a comprehensive technology solution with multiple applications that is redefining document processing, data management and accessibility in various industries and fields. These are the main reasons why the objective of this project is to attempt the implementation of a character digitization system to a calendar that has been annotated with commitments corresponding to specific days of a specific month of the year. In order to achieve this, it is necessary to create a pipeline that can be employed to digitize the content, assign it to a category and store it in a database or in an existing digital calendar, for example using the Google Calendar API or the iOS Calendar API.

Challenges The process of human handwriting recognition and its subsequently digitization presents a number of significant challenges for an OCR systems. The presence of noise and distortions in images represents a considerable obstacle, as it can negatively impact the efficiency and accuracy of the system. OCR systems may encounter also difficulties in accurately discerning characters when compared with images that are either distorted during scanning or ruined by inherent noise, which could potentially lead to errors in character recognition. Furthermore, the issue of multilingual support introduces another layer of complexity, as OCR systems may face challenges in processing documents that contain multiple languages, each with its own set of characters and linguistic rules. It's important to point out that, just like a human, the type of characters the system is asked to recognize is closely tied to a specific alphabet. The system learns the local features of the different characters directly from

the handwritten text, there are input data in the form of $I = (x_i, y_i)$, $y_i = \{y_{(i,1)}, y_{(i,2)}, \dots, y_{(i,N)}\}$, where each $y_{(i,j)}$ is a symbol, or **grapheme** (from gr. $\gamma\rho\alpha\phi\omega$, 'write'), mapped in a digital encoding and decoding alphabet. The system learns a specific alphabet directly from the text and since local features (shape, thickness, corners, edges,...) are inherent to the characters of an alphabet, the system suffers a significant performance degradation when applied to characters outside its known alphabet. This significantly limits the **transfer learning techniques**, since the network must be retrained from scratch to recognize another alphabet different from the one on which the model has already been trained.

Moreover, the diversity of handwriting styles further complicates the OCR process. Handwriting is a highly personal characteristic and considerably differs from person to person: therefore, OCR system has not only to accommodate individual variations, but has also to deal with unconventional handwriting styles (i.e artistic fonts, and the complexity of cursive writing). The inherent subjectivity of handwriting makes it imperative to develop an OCR system that is robust: it should handle as much as possible the actual changes in font and style from text to text and decipher the myriad ways in which individuals express themselves on paper. Taking the Latin alphabet into account, we can differentiate the characters into **capital letters** and **lowercase letters**. It is important to note that while capital letters show less variation in character representation, lowercase handwriting, although it is constrained to some basic calligraphic guidelines, involve more the problem of personal handwriting and, consequently, the distinction in different **calligraphic styles**.

2. Related Works

The first attempt to digitize text was undertaken with the advent of LeNet [4], which demonstrated for the first time the capacity of a shallow convolutional network to accurately recognise numeric digits of the size 32×32 . Some years later, Xu et al. propose a working methodology based on individual MNIST characters dataset [6] to tackle a little bit more complex form of handwriting recognition. The study provides a basic working pattern methodology, especially regarding the section of image pre-processing (involving grayscale pixel uniformation, image cropping and resizing), that can be applied to recognize a bunch of individual character images. Their research objective is to highlight the advantages of using a CNN, Convolutional Neural Network, to extract local features from images. The CNN-based approach will be widely used in last year's bibliography by lots of authors: In 2015 Shi et al. proposed a slightly more complex model

called CRNN. It is a combination of two architectures: a CNN is used to learn the local features of characters from the image while a RNN is employed to model a sequence of symbols, in this case individual characters, to form a word. RNNs are particularly useful as they model a kind of memory and are in fact able to **process sequences** even if they suffer what is so called the vanishing gradient problem. In 2020 Bora et al. present an handwritten character recognition system that combines CNN with another classifier called by the author ECOC, Error Correcting Output Codes. CNNs model like LeNet [4] and AlexNet [9] are used for feature extraction, while ECOC is used for the classification. This component is trained using SVM, Support Vector Machine, where one class is taken as positive in contrast with all the other that are taken as negative, diving the multi class problem into several binary problems. The experiment prove that the CNN-ECOC approach achieves higher accuracy compared to traditional CNN classifiers with a simple softmax layer and is still conducted using MNIST [6] dataset. One year later, Jebadurai et al. described a wider more complex deep learning model based of CRNN to recognize handwritten documents and convert them into digital text, based not only on CNNs to identify the features of human handwriting, but also on LSTMs, Long Short-Term Memory, to understand the sequence of characters that constitute a certain word/phrase. This application use a particular type of loss function called CTC [11], Connectionist Temporal Loss, that is particularly usefull to train model that needed to predict an output sequence having its ground label but without having explicit references to its segmentation. CTC works by attempting all possible placements of characters, taking the sum of all the probabilities, and so allowing the model to predict the text for the image without requiring to know explicitly character placement in the input image. To go beyond this model it has been waiting an application of the transformer [12] to a computer vision, represented by the advent of ViT, Visual Transformer [13].

It is worthy to point out that, before the advent of CRNN and ViT model, the majority of the proposed achieved high performance in term of recognition accuracy but showing huge limitations: the input for the first OCR neural model was necessarily provided in the form of individual character alphabet, the networks were been able to classify the salient features of the character and provide a classification consisting of the corresponding digital label, but with inability of creating a context (both in term of previous word in sentence both in terms of individual characters in a single word). and thus to don't recognize strings of characters.

3. Implementation Description

In order to ensure the accurate digitization of the calendar, the overall working pipeline is composed of several neural blocks. The initial neural component of the pipeline is a **segmentation network** constructed on top of the U-Net [14] architecture. This stage allows the model to learn to identify the most relevant semantic regions of the calendar, thus enabling their subsequent algorithmic extraction through the generation of a segmentation mask. Once the calendar segments have been retrieved from the image, it is possible to proceed with their digitization using the remaining components of the system. Two residual convolutional neural networks, ResNet6 [15], are employed, respectively, for the recognition of the month and days. It is worthy to note that while the recognition of the day digits is performed on more standardised data (given that the same digits always repeat in the same manner across the entire dataset), slightly different considerations must be made for the month. Indeed, the users who decide to record their commitments on the calendar are permitted to write the month on their own and this behaviour can often result in inconsistencies in both the style of writing and the positioning of the written word within the month slot of the calendar. To mitigate the negative effects of these inconsistencies and to facilitate the classification process, a different couple of AprilTag [16] for each year's month has been incorporated into the calendar template, placed at the left and right edges of the month slot. The objective of this design choice is to induce more uniform features in the images, and therefore in the convolutional process, to compensate to a limited extent for the difficulties introduced by the user's choices regarding the management of the calendar title. The final step of the working pipeline is a second U-Net network applied to the reminders fragments extracted from the calendar. The process entails the segmentation of the English text sentences written by the user into their single words which are then conveyed to the neural block designated for the digitization of the user's note. This digitization neural block is constituted by the combination of the visual transformer, ViT [13] and BERT [17].

An additional feature that makes the project more interesting is the possibility of classify the content of each single text line by using the LLM BERT for final text classification task. With an additional effort is also possible to bring the classification at the level of the entire calendar stacking together all the features extracted from the single lines coming out from BERT and then, calculating their mean, obtaining a single features vector that will be then responsible of the assignment of the most probable class for the entire content of the calendar.

Dataset	Training	Validation
Calendar	146	74
Numeric Digits	682	527
IAM Lines/Sentences	7561	3781
IAM Words	64302	32152

Table 1

Datasets: the calendar dataset has been created manually by using a graphic tablet, numeric digits is a collections of numbers in the range $[1, 31]$ generated from computer system fonts, while IAM (Institut für Informatik und Angewandte Mathematik) Handwritten is a dataset belonging to the University of Bern.

3.1. Dataset

To conduct the experiment I have to composed autonomously a dataset of handwritten calendar images, made by the auxiliary of a graphic tablet. The corresponding ground truth segmentation mask have been created using the Oxford VGG Software². The resulting dataset is not particularly huge (see Table. 1) mostly for the reason that with respect to other segmentation task that request to handle a set of heterogeneous images (e.g **ImageNet**) the actual case of study present data that can be said to be homogeneous with respect to their content, indeed the subject is always the same across all the input images, made by a symmetric and well defined structure. Moreover, on the images has been applied an additional works done with **Photoshop** software in a such a way to induce parallax defects (horizontally, vertically or both): this is done because, thinking of the system as an integrated camera software, it is not always possible to guarantee that the image would be acquired with a frontal perspective perfectly aligned with the subject that needs to be scanned. Acting this way the model can be then made more robust with the addition of a **RANSAC** [18], Random Sample Consensus, system that, exploiting the presence of the four **AprilTag** positioned at the extreme corners of the calendar sheet, can realigns the subject acquired via the camera with the original calendar template.

The numeric digits dataset is assembled by saving pictures of numbers from 1 to 31 using classical fonts available at the level of the computer's operating system. This choice will result usefull both because it leads to the training of a small ResNet for the calendar days recognition both because the model could go beyond the font choosen for the calendar in the study case experiment, allowing potentially the possibility of create new calendars with new fonts style for the day numbers, with the advantage of still having a robust model to process them.

Other useful data are retrieved from the public Inter-

net. In particular, the IAM Handwriting Datasets [19] are used to make the pre-trained of the Word Paser model and of the Note Digitizer. IAM Handwriting is a dataset that contains a collection of words cut from portions of written texts, and the main advantage of its using, as analyzed in the introductory section, is the possibility to train a model that learns the relevant features of the individual characters directly from the word image. This approach enables the system to learn the sequential features of the characters that together make up a particular word.

3.2. Data Processing

Standard computer vision techniques are employed to process the images contained in the datasets: the Gaussian blurring is preferred to a classical blurring, since using a kernel that comes from a Gaussian distribution helps to attenuate image noise and details, ensuring to better preserve edges and contours. The image pixels intensity values are uniformed by applying a binary threshold technique that sets to a maximum value all that pixels above a certain threshold (represented for example in the case of IAM dataset by the gray scale levels noted in the data), while putting the values below to 0. Moreover all the pixels values of the image are additionally scaled by a factor $1/255$ to put them in the interval $[0, 1]$ and normalized around a certain mean a standard deviation. This general steps ensure to have data in a format which can be better processed by the models.

Regarding the calendar dataset, to ease the computational segmentation process, images are downsampled using a Laplacian Pyramid. The reason why this technique is employed on the the calendar data before feed them into the network will result more clear in section 3.4.1.

It is worthy also to notice that since a word segmentation block for the calendar note segments has been included into the architecture, it needs to be trained on couple (image, word mask) that are not directly available in the dataset. This is why a dedicated image processing is applied online during the dataset creation to obtain from a single text line image its corresponding ground word segmentation mask: the original image, Figure. 1a is filtered with a Laplacian operator in order to obtain its details, border and edges, Figure. 1b. This operation is particularly useful in the current data domain because, since as previously said in Section. 3.1, the images coming from IAM Line/Sentence datasets are nothing more than a collections of words/text lines cuttings; By applying a further filtering on pixels values of the details, it is possible to highlight border and edges and consequently the cuttings borders with respect to their content, 1c. At this point, by applying a binary inverse threshold function it is possible to obtain the final word binary segmentation

²Oxford VGG, Visual Geometry Group, https://www.robots.ox.ac.uk/~vgg/software/via/via_demo.html

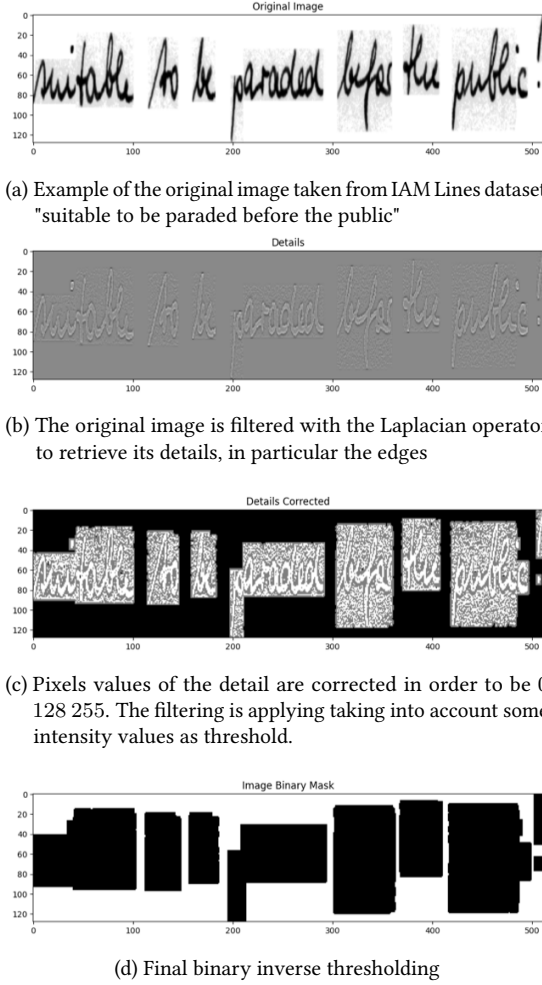


Figure 1: IAM Lines Mask Creation: since each sentence contained in the IAM lines dataset is constructed as a collage of words cuttings, there's a gray scale level provided that denote the break between the white background and the "internal" background of the little piece of word. This information can be exploited to construct more or less accurate binary word masks to trained a second **U-Net** to learn the text line segmentation.(1d)

mask, Figure. 1d.

3.3. Neural blocks

This section explain more in detail the singular neural blocks that constitute the entire digitization system. The system is composed of five neural blocks that work simultaneously:

Calendar Parser It consist of a **U-Net** model, composed of an encoder-decoder equipped with skip connections in the middle. It takes as input an image of the calendar, scaled by using a Laplacian Pyramid, and classifies each of its pixel with a value that denotes its the belonging class. The number of the classes admitted by the model is equal to number of segment in which the image should be segmented. In this case of study 6 has been chosen to be the number of classes for the calendar segmentation. It will be mentioned later on in the Section. 3.4 how in a first moment the number of classes chosen for the experiment was effectively more huge than the final one. It was an attempt to obtain an output class that could model the digital content of the month and day segments directly at the level of the calendar segmentation.

Word Parser It consist of a second **U-Net** model that is pre-trained with couples (image, mask) from IAM Lines/Sentences dataset, with the mask generated as seen in the Section. 3.2, and attempts to predict the most suitable binary segmentation mask for the text line image. As it will be shown in Section. 4, this components demonstrates to be very good in the **word segmentation** of the text lines of the calendar, even if the images differs a lot from the ones used during its training/validation phase.

Month Digitizer It consist of a tiny **ResNet** architecture (just 6 block with 2 residual connections) that work directly on the month segment retrieved from the calendar thanks to the help of the Calendar Parser. Its objective is to classifies the incoming data into 12 possible classes (one for each year's month). Its architecture follow the schema presented in Table.2.

Day Digitizer Similar to the Month Digitizer, it consists as well of a tiny convolutional **ResNet** that is pre-trained over the Numeric Digit dataset and later on fine tuned directly on the days segments coming from the calendar segmentation step. Its final layer is made of a Linear classifier composed of 31 classes (one for each day of the month). Its architecture follow the schema presented in Table.2.

Note Digitizer It consist in a Vision Encoder-Decoder Model, composed of an Encoder modeled as **ViT** which is responsible of the path exctraton from the incoming input and its projection on an higher dimension encoding latent space. **BERT** is instantiated as the decoder and it is responsible of taking the latent space of the encoder and map it to an output label, that in this case is represented as a sequence of inividual character that constitute the final word.

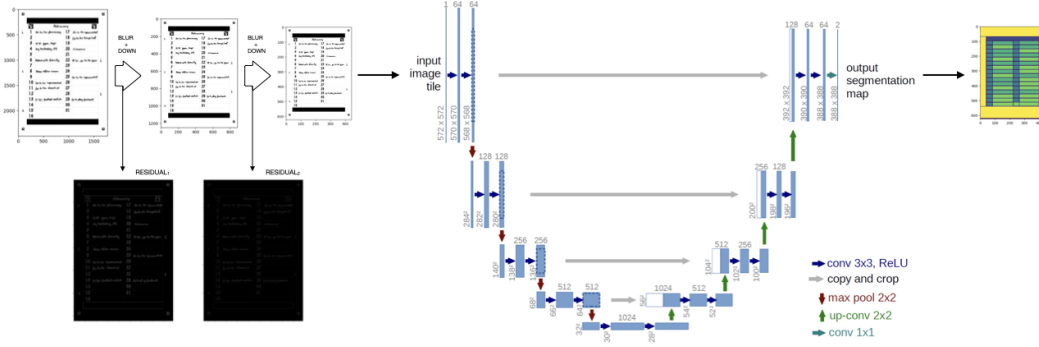


Figure 2: Calendar Parser: it represents the first neural block of the overall digitization system. The model, represented by a **U-Net** network takes as input a downsample image of a calendar and predict as output its segmentation map. It worthy to note that the input image is scaled by using a Gaussian and Laplacian Pyramids both for two reason: 1) providing an input that is not too much big for the model in order to help its computations. 2) Once the calendar fragment are retrieved thank to segmentation map is important to bring them back to the original sizes by adding their corresponding residuals stored from the pyramid downsampling process. This ensure to have images that are poor in resolution for the next digitization steps.

3.3.1. Content classification with BERT

Once the handwritten text has been digitized it may be interesting to classify it into categories in order to **high-light the nature of its content**. To do this it is possible to rely on Transformer-type architecture modeled by a modern LLM such as BERT. BERT is a transformer type architecture consisting of a single encoder. Its peculiarity lies in the fact that given the nature of its training (NSP and Q/A problems) it is easily adaptable to a new working domain simply by fine-tuning the encoder on the new data set and and stacking an additional classsification layer on top of its head. BERT incorporates an excellent semantic knowledge of the English language and therefore can be used to process numerous types of textual content.

Content Text Classifier To conduct the classification of our text, first of all it is necessary to transform each word of the sentence into its corresponding lowercased so as to reduce the variance on the data and filter the textual content by removing what in NLP terminology is defined as "**stopwords**" (i.e. pronouns, articles, etc.). By doing this last step, it is possible to obtain a cleaned text without redundant elements, leaving only those specific words that refer to one topic rather than another. BERT assigns each token of the sentence to a latent vector of size 1×768 . The output of the transformer is therefore a matrix of size $L \times 768$ and a special latent vector, [CLS], of size 1×768 , which encodes the general meaning of the sentence. The [CLS] **context token** is particularly useful in the following use case because when it is needed

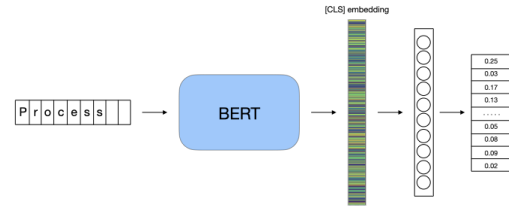


Figure 3: Text Classification Component: relying on BERT LLM architecture it is mainly composed of a fully connected that act as a classifier of [CLS] special token features. CLS BERT special token embed the overall meaning of the entire sentence resulting very suitable to classify the text's content.

to perform content classification, it makes sense to rely on the general meaning of the text rather than the individual tokens that make it up. The model is trained by reducing the MultiClass Cross Entropy loss and applying a Softmax σ to create a probability distribution over the output content labels:

$$l_i = \sigma(W_c \cdot g(x_i)) \quad (1)$$

W_c is the classifier, modeled as a Fully Connected Linear layer, $g(x_i)$ represents the application of the BERT architecture to the input, as each input token is mapped into the encoder latent space with a $H_{1 \times 768}$ latent vector to represent its textual features content.

Layer	In	Out Chann	K	S	P
Linear FC	32	12 or 31	-	-	-
AvgPool	-	-	-	-	-
Batch Norm 6	32	-	-	-	-
Conv 6	32	32	3x3	1	1
Batch Norm 5	32	-	-	-	-
Conv 5	32	32	3x3	1	1
Batch Norm 4	32	-	-	-	-
Conv 4	16	32	1x1	2	-
Batch Norm 3	32	-	-	-	-
Conv 3	16	16	3x3	1	1
Batch Norm 2	16	-	-	-	-
Conv 2	16	16	3x3	1	1
MaxPool2D	-	-	3x3	2	1
ReLu	-	-	-	-	-
Batch Norm 1	16	-	-	-	-
Conv 1	1	16	7x7	2	3

Table 2

Parameters setting for the neural components Month Digitizer and Day Digitizer. Both the neural components are constructed on top of a reduced form of **ResNet** for computational reasons. Starting from the bottom the architecture presents a first convolutional layer, followed by the first residual block (constituted by two convolutional operations separated by a batch normalization in the middle) that sums its output with the one coming from the previous layer (residual connection). Then there's a downsampling operation performed by the 4th convolution with kernel (1x1) followed by the second residual block and at the end on top there's an aggregation layer made with average pooling followed by the final Linear classification layer, that is fully connected and presents a number of output features of 12 and 31 respectively for the Month and Day Digitizers.

3.4. Experiment

The overall project is made up of a bunch of experiments that have been conducted on the different architectural components examined in Section 3.3: since both the calendar parser and the word parser take benefit of using U-Net segmentation architecture, both the model has been trained using **Dice Loss** [20]. The Dice-Sørensen coefficient, Equation 2, also known as Dice Coefficient, is a statistic used to gauge the similarity of two samples. It is widely employed in the field of segmentation, where it is particularly effective in evaluating the pixel-wise accuracy of ground truth masks in comparison to the output segmentation mask generated by a model.

$$Dice(P, G) = \frac{2|P \cap G|}{|P| + |G|} \quad (2)$$

P and G are the set of pixels belonging respectively to the predicted and the ground truth masks, $|P \cap G|$ is the number of pixels that both sets have in common

and finally $|P|$ and $|G|$ represent the total amount of pixel in the two sets. The calendar parsing is essentially a MultiClassification task. The Equation 2 becomes:

$$Dice(P, G) = \frac{1}{C} \sum_{c=1}^C \left(1 - \frac{2 \sum_{i=1}^N (P_{i,c} \cdot G_{i,c})}{\sum_{i=1}^N P_{i,c} + \sum_{i=1}^N G_{i,c}} \right) \quad (3)$$

the word parsing can be treated instead as a BinaryClassification task and the Equation 2 becomes:

$$Dice(P, G) = \frac{2 \sum_{i=1}^N (P_i \cdot G_i)}{\sum_{i=1}^N P_i + \sum_{i=1}^N G_i} \quad (4)$$

P_i is the predicted binary value for pixel i and could be $P_i = 1$ or $P_i = 0$, G_i is the same but in the ground truth mask while N is the total number of pixel, and since the task deals with 2D images, $N = W \times H$, where W and H are respectively the width and the height of the images. The value of the Dice Loss can be easily obtained as:

$$Dice_{Loss}(P, G) = 1 - Dice(P, G) \quad (5)$$

Looking at the Equation 5 it can be observed that minimizing the loss means effectively maximizing the Dice Coefficient that provides a measurement of the similarity between the model's prediction and the ground truth. Once the model is trained, the most suitable prediction of the segmentation map is obtained respectively as $P^* = \argmax_c (P_{i,c})$ in a multi-class domain and in the simpler binary case as $P^* = 1 (P_i \geq 0.5)$, with 1 that is an indicator of whether the condition is satisfied or not.

3.4.1. Design Choices

The images are converted from 3 channels (RGB) to 1 channel (Black and White) and processed as mentioned in the Section 3.2. Calendar images are downsampled using a Laplacian Pyramid before feeding them into the network. This is a very useful technique and it is used since at each blurring-downsample step of the pyramid it is possible to retain the **residuals** of the image (see bottom left in the Image 2). Once the calendar is segmented, for each segment we're interested in, it is possible to upsample and sum it up with its matching area in the residual pyramid to bring it back to the original resolution. This design choice will result particularly useful to deal with the problem of image resolution that otherwise risk to be too much small to be digitized. Regarding the resize operation, days images are brought to 32x32 while month calendar fragments once retrieved are extended to 128x512. It is worthy to mention also that since a ViT [13] is employed in the note digitization phase, word images coming from IAM Words dataset are resized to be 224x224. This dimension ensures the visual transformer

to properly apply it first convolutional step that involves the extraction of images patch with a kernel of 16x16 and a stride of 16x16.

One last mention is about VisualEncodeDecoder model: HuggingFace provide a class to wrap together a visual transformer with a LLM as decoder. Instead of relying to the pre-trained TrOCR [21], I’ve instantiated a new model ensuring also to modify the fist convolutional layer of ViT to adapt the input channels to the condition of the current experiment. This is done because ViT is trained on ImageNet [22] and is used to process three channels input while in this case of study, the input channels have been reduce to one both to ease the computational process of the models both because since we’re dealing only with calligraphy, that is generally presented as grayscale for each channel in RGB mode, there’s no useful information that can be extracted from all the 3 channels.

3.4.2. Training Strategy

The models Calendar Parser and Word Parser are trained respectively on calendar dataset and IAM lines dataset in order to develop the capability of producing a suitable segmentaion map as output. For both the models is employed the Dice Loss (see Section. 3.4) in the back-propagation phase to update the weights of the model and the overall learning process is optimized by Adam with an initial learning rate tuned in the range of 0.001 to 0.0001. The batch size of the data for the calendar parser has been set to 4 while for the word parser the best configuration is reached with a batch size of 8 with gradient accumulation set 16 to simulate a larger batch size and help the model to better generalize its predictions.

The Day Digitizer model is trained over the numeric fonts dataset and fine tuned over the day segment obtained by the calendar segmentation process while the Month Digitizer is trained on a bunch of month segments retrieved by the calendar segmentation and that are heavy augmented to induce more robustness against the issue related to month calendar stop discussed in Section ?? Both employs Adam as optimizer during the learning process and are trained with an initial learning rate of 0.001 and a batch size of 4.

Instead, the Text Classifier is trained with an Adam optimizer, with a learning rate of 0.00015 and a batch size of 32, accumulating the gradient every 32 steps, simulating an effective size of 32×32 . This practice is used with great success in many NLP tasks where, despite the limited hardware available, it is crucial to have higher batch sizes so that the model can better distinguish between different types of data, significantly increasing its performance.

Table 3

Results of the training achieved by the singular neural components: the best candidate for the calendar segmentation is the one with 6 segmentation classes as output, the month e day digitizers both show great capability in the recognition and so in the classification of the calendar fragments extracted with the segmentation map, word parser is pre-trained over the IAM Lines dataset and becomes good in performing the segmentation of the text line into word pieces and the note digitizer has acquire a good level of digitazion capability even if it should reach a little bit greater performance in terms of CER (Char Error Rate) and WER (Word Error Rate).

Experiment	Valid F1	CER	WER
Calendar Parser 6	0.986	-	-
Calendar Parser 37	0.906	-	-
Calendar Parser 49	0.942	-	-
Month Digitizer	0.937	-	-
Day Digitizer	0.978	-	-
Words Parser	0.986	-	-
Handwriting Digitizer	-	0.201	0.272

Table 4

Text classification with BERT LLM: The skills acquired by BERT during its pre-training make it a perfect candidate for text classification. The model can be easily adapted to the new domain with the fine-tuning, and its special context token (i.e. [CLS]) allows us to achieve a good level of accuracy in text classification after just few training epochs.

Text Classification	Train F1	Valid F1
BERT Text Classifier	0.84	0.77

4. Results

In the current section it is discussed the result of the experiment highlights its strengths and weakness. The results of training the calendar parser and the other neural block are reported in Table. 3 while text classification is reported in Table. 4.

4.1. Calendar Segmentation

The model performs the segmentation of the calendar in a satisfactory manner. Figure. 4 illustrates the six distinct binary masks that are generated when a specific classification segment is requested from the model. The class 0, which corresponds to the None/Contours category, is defined during the training phase due to the fact that, during the manual segmentation of the calendar for the generation of its ground truth mask, some segments do not overlap with each other, thus leaving holes inside the mask. Class 5 corresponds to the borders of the calendar and is established with the objective of ensuring that the model’s focus is on the most relevant features, namely

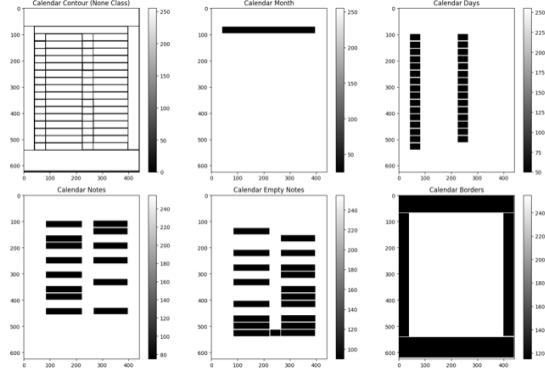


Figure 4: Result of the segmentation process performed on the calendar image. Each binary mask correspond to a particular segmentation value, in the range $[0, 5]$

those pertaining to one-month, two-day, and three-note segments. The class 4, which corresponds to empty notes, was incorporated into the segmentation map with the objective of eliminating, during the digitalisation phase, all those areas whose average pixel values assume a value close to 1, which corresponds to white. In this instance, it can be inferred that for the specified day, the user hasn't notes any reminders.

4.2. Text line segmentation

The Word Parser achieves good accuracy in creating binary segmentation masks for dividing the sentence image into single words. The result of this operation will be shown in the Appendix. ??, where we can note that, although the process of creating binary masks reported in section 3.2 may not be very accurate in some cases, the model was still able to find segmentation patterns among the various training samples that in some situations allow it to predict a mask that is even better than the one that was generated algorithmically and assumed as ground truth.

4.3. Calendar fragments digitization

The calendar note digitizer model is initiated by the word pieces that were previously retrieved by the word parser. The word images are initially processed by the encoder, which is responsible for dividing the image into patches, assigning a positional embedding to each patch and projecting this data into its latent space. The latent space of the encoder will be interpreted by the decoder, which will map the encoded latent features with the respective output symbols. In this case study, the decoder is constrained to produce the output in the form of a sequence of single characters, which, when recompiled, will form the final

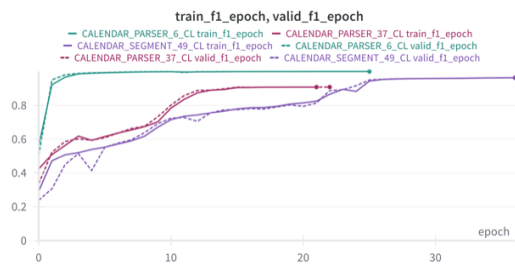
word. This approach, which may appear less efficient than working on entire sentences, primarily considers the fact that an effective context can be extracted from sentences of a certain length. The greater the number of words that precede a current word, the greater the likelihood of identifying semantic patterns that correlate the various words on which the transformer will operate. In the case of the calendar, the sentences are designed to be brief annotations, thus limiting the context available. It is therefore worthwhile to consider the possibility of training the model to learn a mapping between the various image features that could be used to identify an appropriate alphabetic symbol. Acting this way, the model, as shown in Table. ??, achieves a good degree of precision both at the level of the individual characters composing the word and on the word itself. The precision at the character level is calculated as $acc_{char} = 1 - CER$, while that on the entire word is calculated as $acc_{word} = 1 - WER$. Although it performs very well on the dataset used for pre-training, the same cannot be said on our working domain, probably due to differences in resolution and calligraphic style. We conclude so, in order to achieve better performances, it is necessary to extend the handwriting dataset with additional calligraphic styles to make the model more robust possibly applying a little bit more fine-tuning.

4.4. Text content classification

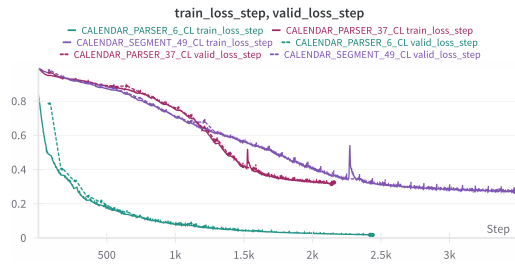
The results of the classification are presented in Table. 4 and appears to be satisfactory, BERT has been demonstrated to be an effective tool for achieving the desired outcome. This is due to its capacity to leverage semantic knowledge of the English language, which is a consequence of its training. The classification was conducted on a domain comprising 15 distinct classes. This can be considered a **fine-grained** classification in comparison to a more aggregated category case, which could be described as **coarse-grained**. Under this hypothesis, the obtained model can be reused to tackle another classification problem without the need to train it again. It is possible to group the micro-classes into meaningful clusters and use this coding to make the classification based on the fine-grain class that leads to a particular coarse-grain cluster.

5. Conclusions

Overall, the experiment provides an overview of how it is possible to tackle a character recognition task through a series of neural models concatenated in a single operational pipeline. The results from the point of view of the segmentation of the calendar and its subsequently the month and day classification are satisfactory. The same



(a) Calendar Segmentation F1 score



(b) Calendar Segmentation loss

Figure 5: Calendar Segmentation Training Results: from the experiment is it possible to conclude that the model that classification with only six segments class as output has the best performance among the three. This is probably due to the fact the the image structure is too standard.

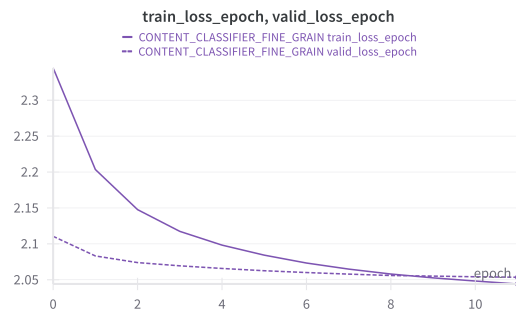
cannot be said for the note digitizer, as it still presents numerous imperfections, both at the Word Error Rate level and at the Char Error Rate level. A good idea would be to provide the model with a more heterogeneous handwriting dataset with more types of calligraphic styles (as even the use of data augmentation techniques did not significantly improve performance on data that completely differ from the training/validation dataset), so as to strengthen the feature detection process and the consequent digitization.

Acknowledgments

The project was carried out as part of the Elective in Artificial Intelligence course of the Sapienza Master Degree in AI and Robotics under the supervision of Professor Christian Napoli. Many thanks also to the Sapienza NLP Group, which provided the dataset used to train the text classification model as part of another course. Finally, thanks to the four students who decided to participate in the final experiment on words. The project files will be made available on the repository <https://github.com/antoogagliardi> after the exam is over.



(a) F1 results



(b) Loss Results

Figure 6: Text classification system training results:

References

- [1] S. Parsons, C. Parker, C. Chapman, M. Hayashida, W. Seales, Educelab-scrolls: Verifiable recovery of text from herculaneum papyri using x-ray ct, 2023.
- [2] E. Stötzner, T. Homburg, J. P. Bullenkamp, H. Mara, R-CNN based PolygonalWedge Detection Learned from Annotated 3D Renderings and Mapped Photographs of Open Data Cuneiform Tablets, in: A. Bucciero, B. Fanini, H. Graf, S. Pescarin, S. Rizvic (Eds.), Eurographics Workshop on Graphics and Cultural Heritage, The Eurographics Association, 2023. doi:10.2312/gch.20231157.
- [3] Y. Fujii, [invited] optical character recognition research at google, 2018, pp. 265–266. doi:10.1109/GCCE.2018.8574624.
- [4] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, in: Proceedings of the IEEE, volume 86, 1998.
- [5] S. Xu, Q. Wu, S. Zhang, S. M. Stanford, Application of neural network in handwriting recognition, 2014. URL: <https://api.semanticscholar.org/CorpusID:16468741>.
- [6] P. Grother, Nist special database 19 handprinted

- forms and characters database, 1995. URL: <https://api.semanticscholar.org/CorpusID:59785963>.
- [7] B. Shi, X. Bai, C. Yao, An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition, *IEEE Transactions on Pattern Analysis and Machine Intelligence* PP (2015). doi:10.1109/TPAMI.2016.2646371.
- [8] M. B. Bora, D. Daimary, K. Amitab, D. Kandar, Handwritten character recognition from images using cnn-ecoc, *Procedia Computer Science* 167 (2020) 2403–2409. URL: <https://www.sciencedirect.com/science/article/pii/S1877050920307596>. doi:<https://doi.org/10.1016/j.procs.2020.03.293>, international Conference on Computational Intelligence and Data Science.
- [9] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: F. Pereira, C. J. C. Burges, L. Bottou, K. Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems* 25, Curran Associates, Inc., 2012.
- [10] J. Jebadurai, I. J. Jebadurai, G. J. L. Paulraj, S. V. Vangeepuram, Handwritten text recognition and conversion using convolutional neural network (cnn) based deep learning model, in: *2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA)*, 2021, pp. 1037–1042. doi:10.1109/ICIRCA51532.2021.9544513.
- [11] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber, Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks, volume 2006, 2006, pp. 369–376. doi:10.1145/1143844.1143891.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *CoRR* (2017). URL: <http://arxiv.org/abs/1706.03762>.
- [13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL: <https://arxiv.org/abs/2010.11929>. arXiv:2010.11929.
- [14] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, *CoRR* abs/1505.04597 (2015). URL: <http://arxiv.org/abs/1505.04597>.
- [15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009. doi:10.1109/CVPR.2009.5206848.
- [16] E. Olson, Apriltag: A robust and flexible visual fiducial system, in: *2011 IEEE International Conference on Robotics and Automation*, 2011. doi:10.1109/ICRA.2011.5979561.
- [17] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding abs/1810.04805 (2018). URL: <http://arxiv.org/abs/1810.04805>.
- [18] M. A. Fischler, R. C. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, *Commun. ACM* 24 (1981) 381–395. URL: <https://doi.org/10.1145/358669.358692>. doi:10.1145/358669.358692.
- [19] U.-V. Marti, H. Bunke, The iam-database: An english sentence database for offline handwriting recognition, *International Journal on Document Analysis and Recognition* 5 (2002) 39–46. doi:10.1007/s100320200071.
- [20] C. H. Sudre, W. Li, T. Vercauteren, S. Ourselin, M. J. Cardoso, Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations, *CoRR* (2017). URL: <http://arxiv.org/abs/1707.03237>.
- [21] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, F. Wei, Trocr: Transformer-based optical character recognition with pre-trained models, 2022. URL: <https://arxiv.org/abs/2109.10282>. arXiv:2109.10282.
- [22] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

A. Month April tags

As reported in Section. 3 to make the classification easier for the month digitizer a couple of April tags is placed on the borders of calendar's month slot. This is going to induce a sort of repetitive features which are different across the twelve months. This ensure that the classification process made by the digitizer calendar doesn't take into account just the calligraphic features that comes from the fact that the user is given the possibility to write the month by itself.

B. Calendar Segmentation Confusion Matrices

Confusion matrices express the ability of the model to align its prediction with the ground truth of each sample. This gives us a visual representation of the percentage of ranked samples with their true value and the classes

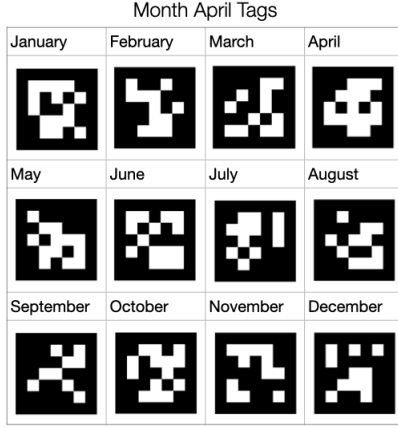


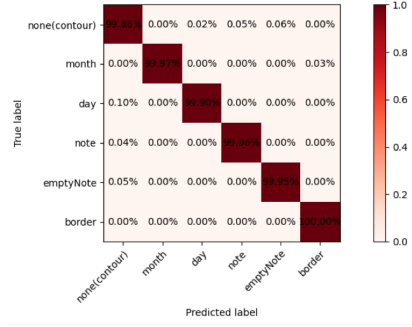
Figure 7: Month April Tags

where the model is most confused. In this way, we can monitor the learning process of the model more closely and also infer empirically which characteristic of the data cause the model to be more confused for certain classes.

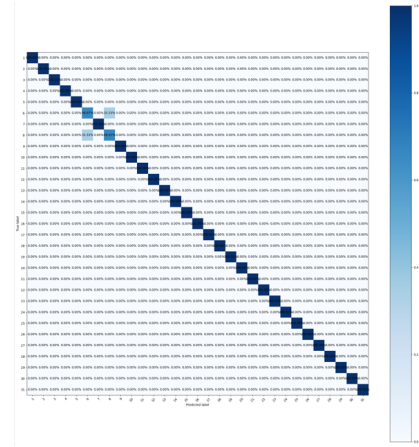
C. Character Error Rate

$$CER = \frac{S_c + I_c + N_c}{N_c} \quad (6)$$

(a) Calendar segmentation confusion matrix



(b) Day fragment classification confusion matrix



(c) Month fragment classification confusion matrix

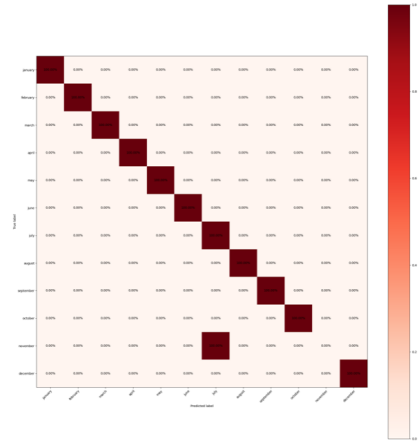
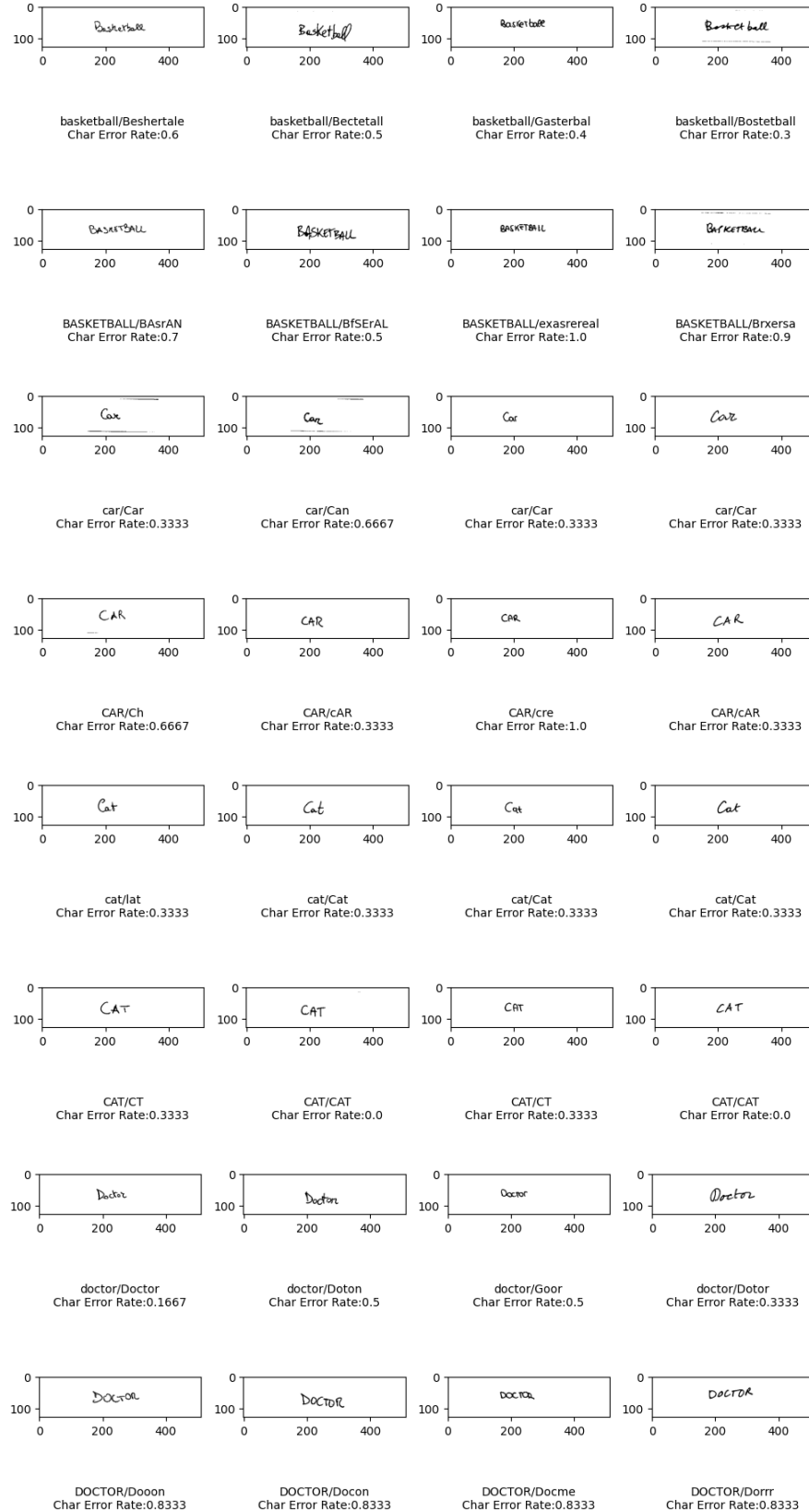
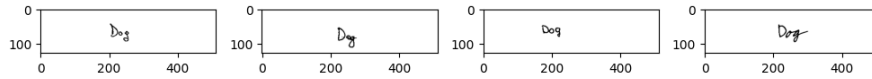


Figure 8: Confusion Matrices results

Figure 9: Word Testing: The OCR model was tested on the handwriting of 4 different test subjects. The table below shows the results of the experiment: In each column we find the subject of the experiment, while the rows refer to a word in lower and upper case that the system had to digitize. The **Char Error Rate** can be used as an additional metric to measure the agreement of the model's prediction with the ground truth corresponding to the word. In this sense, the metric represents a value as a percentage of the number of misclassified characters in the word compared to their actual labels. Technically, it's calculated as the **Levenshtein distance** between the two words, prediction and ground truth. In Eq. [6] S_c is the number of character **substitutions**, I_c is the number of **insertions**, D_c is the number of **deletions**, needed to adjust the prediction to the ground truth. N_c is the total number of characters in the ground truth.



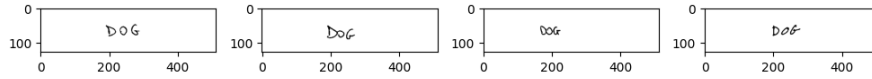


dog/Doy
Char Error Rate:0.6667

dog/Doy
Char Error Rate:0.6667

dog/Doy
Char Error Rate:0.6667

dog/Dog
Char Error Rate:0.3333

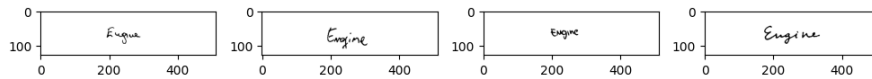


DOG/Doo
Char Error Rate:0.6667

DOG/Doo
Char Error Rate:0.6667

DOG/Dor
Char Error Rate:0.6667

DOG/Doo
Char Error Rate:0.6667

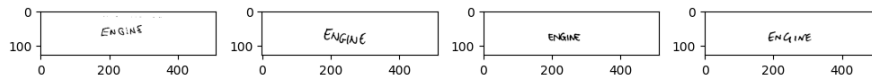


engine/Eugue
Char Error Rate:0.6667

engine/Erngine
Char Error Rate:0.3333

engine/fgne
Char Error Rate:0.5

engine/Engine
Char Error Rate:0.1667

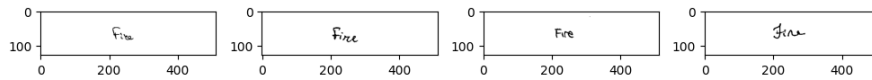


ENGINE/Evsrs
Char Error Rate:0.8333

ENGINE/Enrwe
Char Error Rate:0.8333

ENGINE/evine
Char Error Rate:1.0

ENGINE/Evuure
Char Error Rate:0.8333

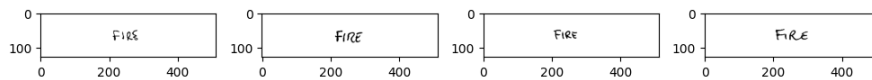


fire/fire
Char Error Rate:0.0

fire/Firne
Char Error Rate:0.5

fire/Fire
Char Error Rate:0.25

fire/Sine
Char Error Rate:0.5

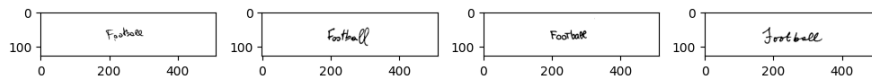


FIRE/fls
Char Error Rate:1.0

FIRE/fine
Char Error Rate:1.0

FIRE/FieE
Char Error Rate:0.5

FIRE/Fire
Char Error Rate:0.75

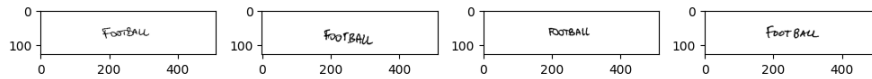


football/Foohll
Char Error Rate:0.5

football/fosthall
Char Error Rate:0.25

football/Fotale
Char Error Rate:0.5

football/tootball
Char Error Rate:0.125

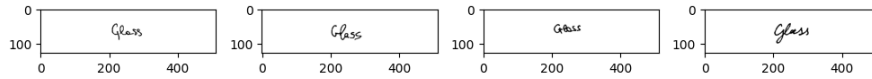


FOOTBALL/forsan
Char Error Rate:1.0

FOOTBALL/foorsu
Char Error Rate:1.0

FOOTBALL/Porsall
Char Error Rate:1.0

FOOTBALL/forom
Char Error Rate:1.0

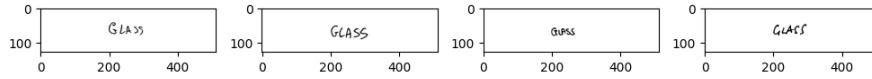


glass/Class
Char Error Rate:0.2

glass/Clss
Char Error Rate:0.4

glass/Gass
Char Error Rate:0.4

glass/Glass
Char Error Rate:0.2

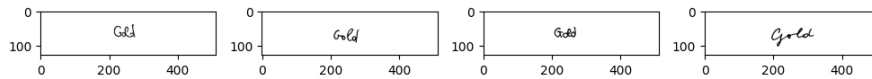


GLASS/OLASS
Char Error Rate:0.2

GLASS/GAss
Char Error Rate:0.6

GLASS/aas
Char Error Rate:1.0

GLASS/Glts
Char Error Rate:0.8

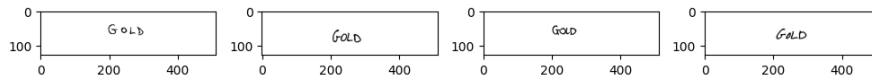


gold/Gad
Char Error Rate:0.75

gold/Gold
Char Error Rate:0.25

gold/Gdd
Char Error Rate:0.75

gold/Gold
Char Error Rate:0.25

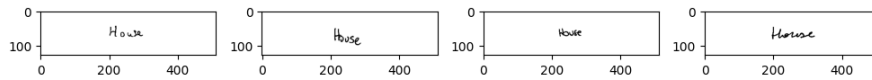


GOLD/Gow
Char Error Rate:0.75

GOLD/Go0
Char Error Rate:0.75

GOLD/Go
Char Error Rate:0.75

GOLD/GowD
Char Error Rate:0.5

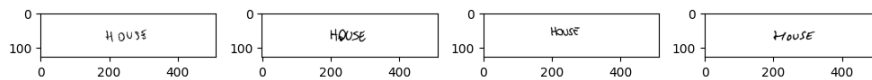


house/Houe
Char Error Rate:0.4

house/House
Char Error Rate:0.2

house/House
Char Error Rate:0.2

house/thouse
Char Error Rate:0.2

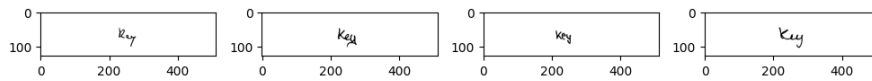


HOUSE/Hous
Char Error Rate:0.8

HOUSE/Hous
Char Error Rate:0.6

HOUSE/Housr
Char Error Rate:0.8

HOUSE/House
Char Error Rate:0.8

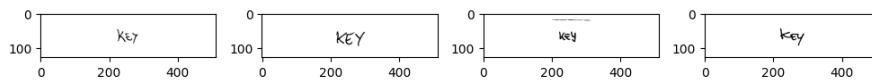


key/hey
Char Error Rate:0.3333

key/key
Char Error Rate:0.0

key/hey
Char Error Rate:0.3333

key/hy
Char Error Rate:0.6667

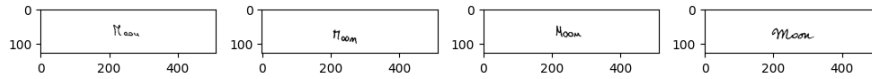


KEY/Kiy
Char Error Rate:0.6667

KEY/key
Char Error Rate:1.0

KEY/Wey
Char Error Rate:1.0

KEY/tey
Char Error Rate:1.0

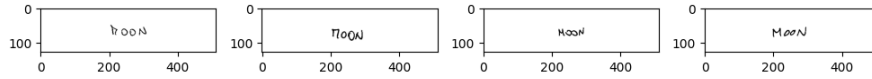


moon/Thon
Char Error Rate:0.5

moon/Mon
Char Error Rate:0.5

moon/Mon
Char Error Rate:0.5

moon/anoon
Char Error Rate:0.5

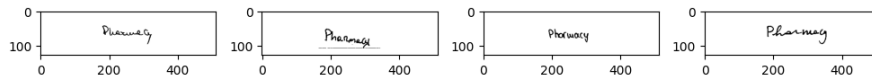


MOON/100N
Char Error Rate:0.75

MOON/nou
Char Error Rate:1.0

MOON/nooN
Char Error Rate:0.75

MOON/Mow
Char Error Rate:0.75

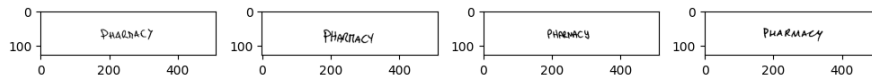


pharmacy/Phanney
Char Error Rate:0.625

pharmacy/Rharnan
Char Error Rate:0.5

pharmacy/Phonacy
Char Error Rate:0.5

pharmacy/Pharmag
Char Error Rate:0.375

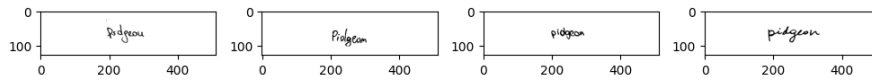


PHARMACY/Pualncy
Char Error Rate:0.875

PHARMACY/PHAc
Char Error Rate:0.625

PHARMACY/ftreny
Char Error Rate:1.0

PHARMACY/PUARIAY
Char Error Rate:0.375

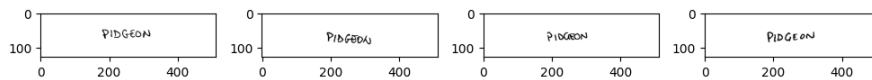


pidgeon/fodgon
Char Error Rate:0.4286

pidgeon/Pidgean
Char Error Rate:0.2857

pidgeon/pidgeron
Char Error Rate:0.1429

pidgeon/pidgen
Char Error Rate:0.1429

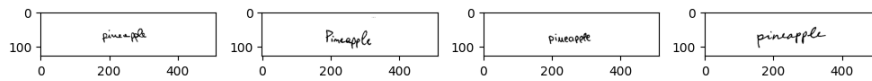


PIDGEON/PIDG0oN
Char Error Rate:0.4286

PIDGEON/P0oow
Char Error Rate:0.8571

PIDGEON/piocron
Char Error Rate:1.0

PIDGEON/Raaeon
Char Error Rate:1.0

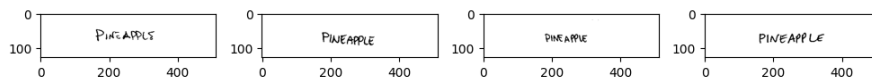


pineapple/fimeapple
Char Error Rate:0.3333

pineapple/Pineapl
Char Error Rate:0.3333

pineapple/pineane!
Char Error Rate:0.3333

pineapple/pineapple
Char Error Rate:0.1111

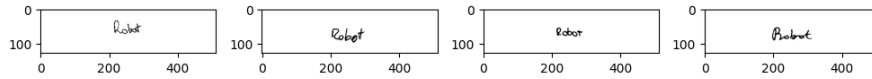


PINEAPPLE/PrEADS
Char Error Rate:0.6667

PINEAPPLE/PwEADLE
Char Error Rate:0.4444

PINEAPPLE/preaRL
Char Error Rate:0.8889

PINEAPPLE/puseARLE
Char Error Rate:0.6667

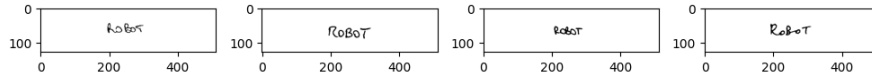


robot/Robt
Char Error Rate:0.4

robot/Rabot
Char Error Rate:0.4

robot/Roor
Char Error Rate:0.6

robot/Rabot
Char Error Rate:0.4

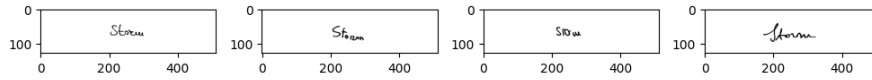


ROBOT/Lobor
Char Error Rate:1.0

ROBOT/ROBOT
Char Error Rate:0.2

ROBOT/Roor
Char Error Rate:0.8

ROBOT/Robor
Char Error Rate:0.8

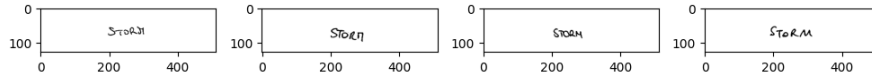


storm/Socn
Char Error Rate:0.8

storm/Stomn
Char Error Rate:0.6

storm/srom
Char Error Rate:0.4

storm/Srom
Char Error Rate:0.6

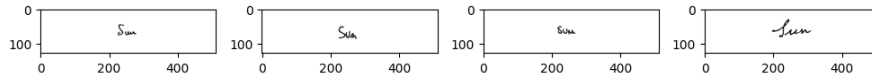


STORM/srols
Char Error Rate:1.0

STORM/sroen
Char Error Rate:1.0

STORM/sroen
Char Error Rate:1.0

STORM/Srkn
Char Error Rate:0.8

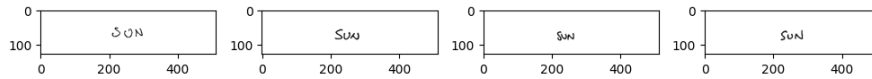


sun/Sm
Char Error Rate:1.0

sun/Sm
Char Error Rate:1.0

sun/sm
Char Error Rate:0.6667

sun/Jun
Char Error Rate:0.3333

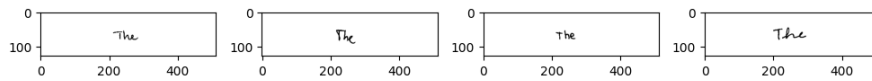


SUN/sN
Char Error Rate:0.6667

SUN/sw
Char Error Rate:1.0

SUN/sow
Char Error Rate:1.0

SUN/sur
Char Error Rate:1.0

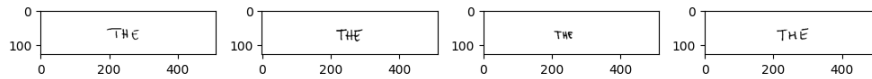


the/The
Char Error Rate:0.3333

the/The
Char Error Rate:0.3333

the/The
Char Error Rate:0.3333

the/The
Char Error Rate:0.3333



THE/TE
Char Error Rate:0.3333

THE/THE
Char Error Rate:0.0

THE/TE
Char Error Rate:0.3333

THE/THE
Char Error Rate:0.0

